

CAPSTONE PROJECT REPORT

SELENIUM PYTHON AUTOMATION FRAMEWORK

PROJECT TITLE 2

E-COMMERCE WEB APPLICATION AUTOMATION USING SELENIUM

Submitted By

Name: Amit Raj

Enrolment No.: 12023052018002

Course: Python Automation

Stream: Computer Science and Business Systems (CSBS)

Semester: VII

Organization: Wipro

Project Details

Automation Tool: Selenium WebDriver

Programming Language: Python

Testing Frameworks: PyTest and unittest

Design Pattern: Page Object Model (POM)

Project Type: Web Application Test Automation

Certificate

This is to certify that **Amit Raj** has successfully completed the study and practical work documented in this report on “**Selenium Python Automation Framework**” as part of a **Python Automation placement-drive training course**.

The project demonstrates the practical implementation of **Python, Selenium WebDriver, PyTest, Unittest, Page Object Model (POM), Utility Classes, Configuration Management, CSV-based Test Data Handling, Screenshots on Test Failure, and HTML Reporting**.

Faculty Name: Anindita Das Bhattacharjee

Date: 25 September 2026

Declaration

I hereby declare that this report titled “Selenium Python Automation Framework – Capstone Assignment 2” has been prepared by me for academic and training purposes. The project demonstrates the implementation of Python, Selenium WebDriver, PyTest, Unittest, Page Object Model (POM), Utility Classes, Configuration Management, CSV-based Test Data Handling, Screenshots on Test Failure, and HTML Reporting.

I confirm that the work presented in this report is based on my understanding and implementation of the assigned project requirements and the learning resources provided during the training.

Name: Amit Raj

Signature: *Amit Raj*

Date: 25 September 2026

Acknowledgement

I would like to express my sincere gratitude to my trainers and faculty members for their valuable guidance, support, and encouragement throughout the development of this Python Selenium Automation Framework.

I am thankful for the learning resources and practical guidance that helped me understand and implement Python, Selenium WebDriver, PyTest, Unittest, Page Object Model (POM), Utility Classes, Configuration Management, CSV-based Test Data Handling, Screenshot Capture on Test Failure, and HTML Reporting.

I also acknowledge the support provided through the training and learning environment, which helped me gain practical knowledge of web application automation and framework development.

Finally, I would like to thank everyone who contributed directly or indirectly to the successful completion of this Selenium Python Automation Framework project.

TABLE OF CONTENTS

- 1. Introduction**
- 2. Project Objective**
- 3. Application Under Test**
 - **3.1 Application Overview**
 - **3.2 Application URL**
 - **3.3 Functionalities Covered**
- 4. Technologies and Tools Used**
- 5. Framework Architecture**
 - **5.1 Project Folder Structure**
 - **5.2 Folder and File Responsibilities**
- 6. Configuration Management**
 - **6.1 Configuration File (config.ini)**
 - **6.2 ConfigReader Utility**
- 7. Test Data Handling Using CSV**
 - **7.1 CSV Test Data Structure**
 - **7.2 CSVReader Utility**
 - **7.3 Benefits of External Test Data**
- 8. Page Object Model (POM)**
 - **8.1 Login Page Object**
 - **8.2 Login Page Locators and Methods**
 - **8.3 Product Page Object**

- **8.4 Product Search Locators and Methods**

9. Utility Classes

- **9.1 ConfigReader**
- **9.2 CSVReader**

10. PyTest Framework Implementation

- **10.1 PyTest Login Test**
- **10.2 Product Search Test**
- **10.3 PyTest Fixture and conftest.py**
- **10.4 PyTest Configuration (pytest.ini)**

11. Unittest Framework Implementation

- **11.1 Unittest Login Test**
- **11.2 setUp() Method**
- **11.3 Test Execution and Assertion**
- **11.4 tearDown() Method**

12. Screenshots on Test Failure

13. HTML Reporting

- **13.1 HTML Report Generation**
- **13.2 Test Execution Report**

1. INTRODUCTION

Software applications require testing to verify their functionalities. Manual testing may become lengthy due to repeated executions of the same test cases. Automation testing is helpful in reducing such manual efforts and providing faster and efficient test execution.

The current project aims at designing Selenium Python Automation Framework for E-Commerce web application. This project framework has been designed in a modular manner such that application actions, test data, configuration, and test cases stay separate.

The current project automates the following important E-Commerce functionalities:

- User Login
- Product Search

Current project demonstrates the implementation of PyTest, Unittest, Page Object Model, and reusable utility classes.

The assignment requirement for the current project includes Unittest, PyTest, POM, utility classes, configuration management, CSV test data, screenshot on failures, and HTML reporting.

2. PROJECT OBJECTIVE

The primary objective of this project is to develop a Selenium Python Automation Framework that will automate the functionalities of an E-Commerce web application.

Objectives of the Project

- Automate the login functionality.
- Automate the product search functionality.
- Using Selenium WebDriver for browser automation.
- PyTest for test execution.
- Unittest-based test execution.
- Page Object Model.
- Reusable utility classes.

- Application configuration to be stored separately.
- Reading of test data from CSV.
- Capture screenshots when the tests fail.
- Create an HTML report for test execution.
- Maintain a clean and scalable project structure.

The project scenario outlined in the assignment is that of automating login and product search functionality using a scalable framework.

3. Application Under Test

3.1 Application Name

The application chosen for this project is TutorialsNinja / OpenCart Demo.

This is an e-commerce website application which provides common online shopping features such as:

- Registration of users
- Login for users
- Search for products
- Browsing for products
- Information about products

In this assignment, we are particularly automating:

- **Login Feature**
- **Product Search Feature**

The assignment clearly states the requirement as automating **the Login and Product Search Feature of an E-Commerce application.**

3.2 Application URL

The application which is used in our framework is the following:

<https://tutorialsninja.com/demo/>

This URL is stored in the configuration file of the project instead of being hard coded in each test.

Our config/config.ini contains:

[application]

url = <https://tutorialsninja.com/demo/>

browser = chrome

This approach allows us to keep the framework well-organized and easy to maintain since in case of a URL change, we do not have to update each test but only the configuration file.

3.3 Functionality Covered

Login

The login test checks whether a registered user can log in the application with the correct credentials.

The automation script performs the following steps:

- Open the application.
- Open the Login page.
- Read e-mail and password stored in CSV.
- Type e-mail.
- Type password.
- Click the Login button.
- Check if the login was successful.

The framework checks if the login was successful using the account URL:

route=account/account

Product Search

The product search test checks whether a user can perform a product search.

The automation script performs the following steps:

- Open the application.
- Read the product name from CSV.
- Type the product name.
- Click the Search button.
- Check if the searched product is present on the results page.

For example, CSV may look like:

email,password,product

your_email,your_password,MacBook

The automation script reads the product name by using:

test_data["product"]

3.4 Why This Application Was Selected

The assignment itself recommends such applications as **TutorialsNinja** or **AutomationExercise** for such a task.

TutorialsNinja was chosen for the task since it offers the required features of an e-commerce application for the assignment:

- Login
- Product search

It also provides a good platform for showcasing the capabilities of Selenium and **the Page Object Model**.

4. Technologies and Tools Used

The framework uses several technologies required by the assignment.

Technology/Tool	Purpose
Python	Programming language
Selenium WebDriver	Browser automation
PyTest	Test execution and test framework
Unittest	Python unit-testing framework
Page Object Model	Maintainable page automation
CSV	Test-data storage
ConfigParser	Configuration management
pytest-html	HTML test reporting
Chrome	Browser used for execution
VS Code	Development environment

4.1 Python

Python programming language is the primary development language for the automation framework.

The language was selected due to the wide support of Selenium and the ability to split the code into separate reusable modules.

For instance, the following code snippet shows how to import the WebDriver module.

```
from selenium import webdriver
```

4.2 Selenium WebDriver

Selenium Web Driver is responsible for the interaction with the browser.

The tool allows

- open and close browser
- open web pages
- locate elements
- type texts
- click elements
- read web content
- take screenshots.

The code snippet below shows how to initialize the web driver and open the website.

```
driver = webdriver.Chrome()
```

```
driver.get(url)
```

4.3 PyTest

PyTest is a test framework that is used for test execution.

Our test files follow the PyTest naming convention:

```
test_login.py
```

```
test_product_search.py
```

```
test_setup.py
```

A typical PyTest test looks like:

```
def test_valid_login(driver):
```

The fixture 'driver' is used to pass the Selenium Web Driver instance.

The following command was used to run the test suite.

```
pytest -v --html=reports/test_report.html --self-contained-html
```

As a result, the test suite execution produces the following report.

4 passed

5. Framework Architecture

Each folder has a specific responsibility.

5.1 config/

This folder houses the configuration information.

File:

config.ini

Which contains:

[application]

url = https://tutorialsninja.com/demo/

browser = chrome

Instead of having to write the URL multiple times in the test files, the framework only has to read it once.

5.2 data/

This folder stores the test data.

File:

test_data.csv

With contents:

email,password,product

your_email,your_password,MacBook

The tests don't actually have the login credentials or product name in the code.

They are read in with CSVReader.

6. Configuration Management

Configuration management is used so that the application specific information is decoupled from the test logic.

Our configuration file is:

config/config.ini

With contents:

[application]

url = https://tutorialsninja.com/demo/

browser = chrome

The framework uses:

utils/config_reader.py

6.1 ConfigReader

The ConfigReader class is used to load the configuration file.

It has methods like:

get_application_url()

and:

get_browser()

For example:

config = ConfigReader()

url = config.get_application_url()

browser = config.get_browser()

The test framework can then do:

driver.get(url)

6.2 Benefits

This approach has a number of benefits.

- Centralized configuration
- Easier maintenance
- Less hard-coded information

- Reusability
- Better framework organization

For example, if we need to change the application URL, we don't have to go searching through every test file. We can simply modify **config.ini**.

7. Test Data Handling Using CSV

The assignment requires **Test Data Handling using CSV**.

The framework therefore uses:

data/test_data.csv

Example structure:

email,password,product
your_email,your_password,MacBook

The CSV contains the input data required by our tests.

7.1 CSVReader

The file:

utils/csv_reader.py

contains the CSVReader class.

The important method is:

read_test_data()

It reads the CSV and returns the data as a list of dictionaries.

The test can then use:

test_data = CSVReader.read_test_data()[0]

and access:

test_data["email"]
test_data["password"]

```
test_data["product"]
```

7.2 Why CSV Is Used

Using CSV provides separation between:

Test logic

and

Test data

For example, the login test doesn't need to contain:

```
email = "..."  
password = "..."
```

Instead, it obtains the values from the CSV.

This makes the framework easier to maintain and allows test data to be changed without modifying the test logic.

Note for documentation: don't expose the real password in your submitted report/screenshots. Mask it, e.g. *****.

8. Page Object Model — POM

Page Object Model is one of the major requirements of Assignment 2.

POM separates:

Web-page elements and page actions

from:

Test cases

This makes the automation code cleaner and more reusable.

Our pages folder contains:

```
login_page.py  
product_page.py
```

9. Login Page

The file:

pages/login_page.py

represents the Login page.

It contains the locators and actions related to login.

9.1 Login Locators

The framework uses:

EMAIL_FIELD = (By.ID, "input-email")

PASSWORD_FIELD = (By.ID, "input-password")

LOGIN_BUTTON = (By.XPATH, "//input[@value='Login']")

These locators identify the corresponding elements on the application

9.2 Login Methods

The page object provides methods such as:

**enter_email()
enter_password()
click_login()**

and:

login()

The combined method:

**def login(self, email, password):
 self.enter_email(email)
 self.enter_password(password)
 self.click_login()**

allows the test case to simply call:

login_page.login(


```
        test_data["email"],
        test_data["password"]
    )
```

This is a good example of POM because the test doesn't need to know the actual locator of the email field, password field, or Login button.

10. Product Page

The file:

pages/product_page.py

contains functionality related to product searching.

The search field is identified using:

```
SEARCH_FIELD = (By.NAME, "search")
```

The search button is identified using:

```
SEARCH_BUTTON = (By.CSS_SELECTOR, "#search button")
```

10.1 Search Methods

The page provides:

```
enter_search_text()
click_search()
```

and:

```
search_product()
```

The combined method is:

```
def search_product(self, product):
    self.enter_search_text(product)
    self.click_search()
```

Therefore, the test can simply use:

```
product_page.search_product(
    test_data["product"]
)
```

Again, the test case doesn't directly deal with Selenium locators.

11. Test Cases

The framework contains tests for the functionality required by Assignment 2.

Test Case 1 — Valid Login

Objective: Verify that a valid registered user can log into the application.

Steps

1. Launch browser.
2. Open application.
3. Navigate to Login page.
4. Read credentials from CSV.
5. Enter email.
6. Enter password.
7. Click Login.
8. Verify successful login.

Expected Result

The user should be successfully logged in and redirected to the account page.

Actual Result

Login was successfully performed.

Status

PASS

12. Product Search Test

Objective

Verify that the user can search for a product.

Steps

1. Launch browser.
2. Open application.
3. Read product name from CSV.

4. Enter product name.
5. Click Search.
6. Verify the searched product is present in the results.

Expected Result

The application should display the searched product.

Actual Result

The searched product was displayed.

Status

PASS

13. Unittest Implementation

The file:

tests/test_login_unittest.py

implements the login test using Python's unittest.

The class is:

class TestLogin(unittest.TestCase):

It contains:

**setUp()
test_valid_login()**

and:

tearDown()

13.1 setUp()

setUp() executes before the test.

It:

1. Loads configuration.

2. Creates Chrome WebDriver.
3. Maximizes the browser.
4. Opens the application.

13.2 test_valid_login()

This method performs the actual login test.

It reads CSV data and uses:

LoginPage

to perform login.

The validation is:

self.assertTrue(login_page.is_logged_in())

If the user is successfully logged in, the assertion passes.

13.3 tearDown()

After the test finishes:

self.driver.quit()

closes the browser.

This prevents the browser from remaining open after execution.

14. Screenshot on Failure

This functionality is implemented in:

conftest.py

The framework uses the PyTest hook:

pytest_runtest_makereport

The logic checks whether the test failed.

If:

report.failed

is true, the framework obtains the WebDriver and captures a screenshot.

The screenshot is saved inside:

screenshots/

using the test name as the filename.

For example:

screenshots/test_product_search.png

Why This Is Useful

When an automation test fails, the screenshot provides visual evidence of the browser state at the time of failure.

This helps with:

- Debugging
- Failure analysis
- Test evidence
- Identifying unexpected UI states

15. HTML Reporting

The framework uses:

pytest-html

to generate an HTML execution report.

The command used was:

pytest -v --html=reports/test_report.html --self-contained-html

The report is generated at:

reports/test_report.html

What the Report Shows

The HTML report provides information about:

- Test names

- Test execution status
- Passed tests
- Failed tests
- Execution details
- Overall test results

The final execution showed:

4 passed in 12.53s

16. Final Test Execution

The complete pytest suite was executed.

Command:

pytest -v --html=reports/test_report.html --self-contained-html

The result was:

tests/test_login.py::test_valid_login PASSED
tests/test_login_unittest.py::TestLogin::test_valid_login PASSED
tests/test_product_search.py::test_product_search PASSED
tests/test_setup.py::test_open_application PASSED

4 passed in 12.53s

Therefore:

Metric	Result
Total tests	4
Passed	4
Failed	0
Pass percentage	100%

The HTML report was also successfully generated.